

Uni-app搜索页面：修复重复搜索未初始化bug

一、bug现象与根源分析

1.1 问题表现

在搜索页面中执行以下操作时出现异常：

1. 首次搜索关键词A，获取并展示正常结果；
2. 未清空输入框，直接修改关键词为B并搜索；
3. 结果区仍残留关键词A的部分数据，或出现数据叠加、页码混乱，甚至无结果时仍显示旧结果。

1.2 根源定位

通过代码排查，确定bug核心原因是**重复搜索时未对关键状态进行初始化**，具体问题点如下：

- **结果数据未清空**：重复搜索时直接覆盖pageNum，但未清空上一次的結果列表，导致新老数据叠加；
- **分页状态残留**：上一次搜索的isLastPage（是否最后一页）状态未重置，若前一次已是最后一页，新搜索会误判为无更多数据；
- **加载状态同步问题**：快速重复搜索时，前一次请求的loading状态未及时重置，导致新请求被拦截。

例如：首次搜索A后pageNum=2、isLastPage=true，修改关键词为B搜索时，未重置isLastPage，导致新搜索直接判定为最后一页，无法加载数据。

二、核心修复方案

修复思路：在每次执行新搜索（含关键词修改后的重复搜索）时，强制初始化与搜索结果相关的核心状态，确保每次搜索都是“全新开始”。

重点初始化的状态包括：结果列表、页码、是否最后一页、加载状态，同时同步重置页面展示逻辑。

2.1 搜索核心逻辑修复（关键代码）

修改“pages/search/search.vue”中的handleSearch与fetchSearchResult方法，补充状态初始化步骤：

Code block

```
1 <script>
2 import { throttle } from '@utils/tools';
```

```

3  import { searchResource, getSearchHotWords } from '@api/search';
4
5  export default {
6    name: 'SearchPage',
7    data() {
8      return {
9        searchValue: '', // 搜索输入值
10       historyList: [], // 搜索历史
11       hotList: [], // 热门搜索
12       suggestList: [], // 搜索建议
13
14       // 搜索结果相关（待初始化的核心状态）
15       resultData: {
16         list: [], // 结果列表
17         total: 0, // 总结果数
18         pageNum: 1, // 当前页码
19         pageSize: 10, // 每页条数
20         pages: 0 // 总页数
21       },
22       showResult: false, // 是否显示结果区
23       loading: false, // 加载状态（防止重复请求）
24       isLastPage: false // 是否为最后一页
25     };
26   },
27   methods: {
28     // 执行搜索：核心修复点—添加状态初始化
29     async handleSearch() {
30       const keyword = this.searchValue.trim();
31       if (!keyword) {
32         uni.showToast({ title: '请输入搜索关键词', icon: 'none' });
33         return;
34       }
35
36       // ===== 重复搜索修复：状态初始化 =====
37       this.initSearchState();
38       // =====
39
40       // 保存搜索历史
41       this.saveSearchHistory(keyword);
42       // 发起搜索请求
43       this.fetchSearchResult();
44     },
45
46     // 初始化搜索状态（独立抽离，便于复用）
47     initSearchState() {
48       // 1. 结果数据初始化
49       this.resultData = {

```

```
50         list: [],
51         total: 0,
52         pageNum: 1, // 重置为第一页
53         pageSize: this.resultData.pageSize, // 保留原每页条数
54         pages: 0
55     };
56     // 2. 分页状态初始化
57     this.isLastPage = false;
58     // 3. 加载状态初始化（防止前一次请求残留）
59     this.loading = false;
60     // 4. 临时隐藏结果区（避免旧数据闪烁）
61     this.showResult = false;
62     // 5. 清空搜索建议（若有）
63     this.suggestList = [];
64 },
65
66 // 接口请求：获取搜索结果（补充状态判断）
67 async fetchSearchResult() {
68     const keyword = this.searchValue.trim();
69     if (!keyword) return;
70
71     // 防止重复请求（初始化后此判断更可靠）
72     if (this.loading) return;
73     this.loading = true;
74
75     // 显示加载动画
76     uni.showLoading({ title: '搜索中...' });
77
78     try {
79         const params = {
80             keyword,
81             pageNum: this.resultData.pageNum,
82             pageSize: this.resultData.pageSize
83         };
84         // 调用搜索接口
85         const res = await searchResource(params);
86
87         // 赋值新结果（无需担心旧数据叠加，已初始化）
88         this.resultData = { ...res };
```

2.2 修复关键点说明

修复点	具体操作	解决的问题
抽离initSearchState方法	将初始化逻辑独立，便于handleSearch、	避免重复代码，确保各场景初始化逻辑一致

	onPullDownRefresh等场景 复用	
结果列表清空	this.resultData.list = []	新搜索结果不会与旧结果叠加
页码重置	this.resultData.pageNum = 1	确保每次搜索从第一页开始加载
isLastPage重置	this.isLastPage = false	避免前一次“最后一页”状态影响新搜索
loading状态重置	this.loading = false	防止前一次请求残留的loading拦截新请求
临时隐藏结果区	this.showResult = false（请求成功后再设为true）	避免旧数据在新结果加载前闪烁

三、关联场景修复与优化

3.1 快速连续搜索场景优化

针对用户快速修改关键词并连续搜索的场景，补充“请求中断”逻辑——若前一次搜索请求未完成，新搜索时中断旧请求，避免数据返回混乱。

Code block

```
1  <script>
2  export default {
3    data() {
4      return {
5        // ...其他数据
6        abortController: null // 用于中断请求的控制器
7      };
8    },
9    methods: {
10     initSearchState() {
11       // ===== 新增：中断前一次未完成请求 =====
12       if (this.abortController) {
13         this.abortController.abort(); // 中断请求
14         this.abortController = null; // 重置控制器
15       }
16       // =====
17
18       // 原有初始化逻辑...
19       this.resultData = { /* ... */ };
20       this.isLastPage = false;
21       this.loading = false;
```

```
22     this.showResult = false;
23 },
24
25     async fetchSearchResult() {
26         const keyword = this.searchValue.trim();
27         if (!keyword) return;
28
29         if (this.loading) return;
30         this.loading = true;
31         uni.showLoading({ title: '搜索中...' });
32
33         try {
34             // ===== 新增：创建请求控制器 =====
35             this.abortController = new AbortController();
36             const signal = this.abortController.signal;
37             // =====
38
39             const params = { keyword, pageNum: this.resultData.pageNum, pageSize:
this.resultData.pageSize };
40             // 调用搜索接口时传入signal
41             const res = await searchResource(params, signal);
42
43             this.resultData = { ...res };
44             this.isLastPage = this.resultData.pageNum >= this.resultData.pages;
45             this.showResult = true;
46         } catch (err) {
47             // 忽略中断请求导致的错误
48             if (err.name !== 'AbortError') {
49                 console.error('搜索接口请求失败：', err);
50                 this.showResult = true;
51                 this.resultData.total = 0;
52             }
53         } finally {
54             this.loading = false;
55             this.abortController = null; // 重置控制器
56             uni.hideLoading();
57             uni.stopPullDownRefresh();
58         }
59     },
60 },
61 // 页面销毁时中断未完成请求
62 onUnload() {
63     if (this.abortController) {
64         this.abortController.abort();
65     }
66 }
67 };
```

同时需修改请求工具类“utils/request.js”，支持传入中断信号：

Code block

```

1  // 修改request方法，支持signal参数
2  export default function request(options, signal) {
3      const baseURL = process.env.NODE_ENV === 'development'
4          ? 'https://dev-api.example.com'
5          : 'https://api.example.com';
6
7      const config = {
8          baseURL,
9          timeout: 10000,
10         header: {
11             'Content-Type': 'application/json',
12             'token': uni.getStorageSync('token') || ''
13         },
14         signal, // 传入中断信号
15         ...options
16     };
17
18     return new Promise((resolve, reject) => {
19         uni.request({
20             ...config,
21             success: (res) => { /* 原有逻辑 */ },
22             fail: (err) => {
23                 // 捕获中断错误并返回
24                 if (err.errMsg.includes('abort')) {
25                     reject(new Error('RequestAborted'));
26                 } else {
27                     uni.showToast({ title: '网络异常，请检查网络', icon: 'none' });
28                     reject(new Error('网络异常'));
29                 }
30             },
31             complete: () => { /* 原有逻辑 */ }
32         });
33     });
34 }
35
36 // 同步修改api/search.js中的调用方式
37 export const searchResource = (params, signal) => {
38     return request({
39         url: '/search/resource',
40         method: 'GET',
41         params

```

```
42     }, signal); // 传入signal
43   };
```

3.2 空关键词与异常输入处理强化

在initSearchState前强化关键词校验，避免无效搜索触发状态紊乱：

Code block

```
1  async handleSearch() {
2    let keyword = this.searchValue.trim();
3    // 强化校验：过滤纯空格、特殊字符等无效关键词
4    if (!keyword || /\s*$/.test(keyword)) {
5      uni.showToast({ title: '请输入有效的搜索关键词', icon: 'none' });
6      // 若输入无效，重置页面状态
7      this.suggestList = [];
8      this.showResult = false;
9      return;
10   }
11
12   // 对关键词进行编码（防止特殊字符导致接口请求异常）
13   keyword = encodeURIComponent(keyword);
14   this.searchValue = decodeURIComponent(keyword); // 保持输入框显示正常
15
16   // 执行初始化与搜索
17   this.initSearchState();
18   this.saveSearchHistory(this.searchValue);
19   this.fetchSearchResult();
20 }
```

四、测试用例与验证

4.1 核心测试用例

测试场景	操作步骤	预期结果
基础重复搜索	1. 搜索“Uni-app”；2. 输入框修改为“mp-html”并搜索	结果区显示“mp-html”相关结果，无“Uni-app”残留数据
连续快速搜索	1. 搜索“A”后立即修改为“B”搜索	A的请求被中断，仅显示B的搜索结果，无数据混乱
前一次为最后一页		

	1. 搜索 “A” 翻至最后一页； 2. 搜索 “B”	B的搜索从第一页开始加载， isLastPage正确更新
无效关键词搜索	1. 输入纯空格；2. 点击搜索	提示 “请输入有效关键词”， 页面状态无紊乱
搜索失败后重新搜索	1. 断网状态搜索 “A” （失败）；2. 联网后搜索 “B”	B的搜索正常初始化，显示正 确结果

4.2 多端验证重点

由于Uni-app多端运行特性，需在以下平台验证修复效果，避免平台差异导致的新问题：

- **微信小程序**：验证请求中断（AbortController）兼容性，部分低版本微信基础库可能不支持，需做好降级处理（可在onLoad中检测基础库版本）；
- **App端（iOS/Android）**：验证loading状态与结果区切换的流畅性，避免原生组件与webview交互异常；
- **H5端**：验证URL编码后的关键词传递正确性，避免特殊字符导致接口400错误。

微信小程序降级处理示例：

Code block

```
1  onLoad() {
2    // 检测微信基础库版本，判断是否支持AbortController
3    const systemInfo = uni.getSystemInfoSync();
4    if (systemInfo.platform === 'devtools' || systemInfo.platform === 'ios' ||
systemInfo.platform === 'android') {
5      const version = systemInfo.SDKVersion || '2.0.0';
6      const [major, minor] = version.split('.').map(Number);
7      // 微信基础库2.10.0及以上支持AbortController
8      this.supportAbort = (major > 2) || (major === 2 && minor >= 10);
9    } else {
10     this.supportAbort = true;
11   }
12 },
13 initState() {
14   // 降级处理：不支持AbortController时跳过中断逻辑
15   if (this.supportAbort && this.abortController) {
16     this.abortController.abort();
17     this.abortController = null;
18   }
19   // 其他初始化逻辑...
20 }
```


五、总结与预防措施

5.1 修复效果总结

本次修复通过“状态强制初始化+请求中断控制+输入校验强化”三重手段，彻底解决了重复搜索时的数据残留、页码混乱等bug，同时优化了快速搜索、异常输入等边缘场景的体验，确保每次搜索都能从“干净状态”开始。

5.2 同类问题预防措施

- **状态管理规范**：将搜索、列表等核心功能的状态抽离为独立模块（如使用Vuex或Pinia），避免组件内状态混乱，便于统一初始化；
- **请求生命周期管控**：对异步请求添加明确的“创建-中断-销毁”生命周期管理，尤其在多轮交互场景中，避免旧请求干扰新操作；
- **通用方法抽离**：将初始化、校验等通用逻辑抽离为独立方法，避免重复编码导致的逻辑不一致；
- **边缘场景测试**：在功能测试中加入“连续操作”“异常输入”“网络波动”等边缘场景，提前发现状态同步问题。

后续开发类似交互功能（如筛选、排序、分页查询）时，可复用本次修复的“状态初始化+请求管控”模式，从源头减少同类bug的产生。

（注：文档部分内容可能由 AI 生成）